Hashing - 1

① **HashMap** → store data in key-value Pair

↳ characteristics:

- **Unique keys**: You can't have Duplicate keys; adding a value with an existing key will overwrite the old value.
- **Unordered**: It does not maintain the order in which elements were added.

op. →
- put(K, V) → insert key-value pair
- get(K) → Retrieves the value for a key.
- contains key (K) → checks if a key exists.
- contains value (V)
- remove (K) → Deletes a key-value pair
- size() → Return the count of pairs
- isEmpty()

Q. freq. of Elements : → key datatype → value datatype

```
<HashMap <Integer, Integer> freq = new HashMap<>();
for (int x : a){
    int count = freq.getOrDefault(x, 0);
    freq.put(x, count+1);
}
```

T.C : O(N+Q)

S.C : O(N)

Integer datatype means using equals() favourable.

Hash Set → store unique keys, without associated values

↳ • add(E) → add element to set if not already present

- addAll(Collection < ? extends E > c)

↳ animals, addAll(dogs) → array list

- remove(o) → remove specific element
- isEmpty()
- size()
- clear() → Remove all Elements from list.

- Count of Distinct element

↳ HashSet <Integer> hs = new HashSet<>();
for (int x : a) {
hs.add(x);
}

T.C = O(N)
S.C = O(N)

Iterating over a HashMap

- Iterate over keys → for (String key : map.keySet()) {
Sop(ln(key);
}

- Iterate over values → for (Integer val : map.values())

- Iterate over key-value pairs

↳ for (Map.Entry <String, Integer> entry : map.entrySet()) variable
{ Sp ln (entry.getKey() + "→" + entry.getValue()

---

order not preserved in HashMap/Hash Set

Minimum Distance Pair

↳ find the min |i-j| such that A[i] == A[j].

public int findMinDist (int[] A)
HashMap <Integer, Integer> map = new HashMap<>();
int minDist = Integer.MAX_VALUE;
for (int i = 0; i < A.length; i++) {
if (map.containsKey(A[i])) {
result = Math.min(result, i - map.get(A[i]))
}
else { map.put(A[i], i); }
if (result == A.length) return -1;
else { return result; }

Hashing-2

Check

① Pair Sum (Two Sum)

↳ Given an array A, and an integer K, check if there exists a pair (i, j) such that $A[i] + A[j] = K$ and $i \neq j$.

Code →
```
HashSet <Integer> hs = new HashSet <>();
for(int i=0; i< A.length; i++){
    if (hs.contains(K-A[i])){ return true;}
    hs.add(A[i]);
}
return False;
}
```

$T.C = O(N)$
$S.C = O(N)$

② Count of Pair Sum

Code →
```
public int CountPairs(int[] A, int K){
    HashMap <Integer, Integer> hm = new HashMap <>();
    int count=0;
    for (int x: A){
        if (hm.containsKey (K-x)){
            count += hm.get (K-x);
        }
        hm.put (x, hm.getOrDefault (x,0)+1);
    }
    return count;
}
```

$T.C = O(N)$
$S.C = O(N)$

③ Subarray Sum Equals 0

```
public int Solve (int [] A){
    int n = A.length;
    long[] P = new long [n+1];
    for (int i=0; i<n; i++){
        P[i+1] = P[i] + A[i];
    }
    HashSet <Long> set = new HashSet <>();
    for (int i=0; i<=n; i++){
        if (set.contains(P[i])) return 1;
        set.add (P[i]);
    }
    return 0;
```

CASE: 1 → P[i] itself 0
CASE: 2 → $P[i] = P[i-1]$
↓
elements added b/w those equals 0.

(4) Longest Subarray with Sum 0

```
public int solve (int[]A) {
    int n = A.Length;
    Long [] Prefix = new Long [n+1];
    for (int i=0; i<n; i++){
        Prefix[i+1] = Prefix[i] + A[i];
    }
    HashMap <Long, Integer> hm = new HashMap<>();
    int maxLen = 0;
    for(int i=0; i<=n; i++ ){
        if (hm.ContainsKey (Prefix[i])) {
            int Len = i - hm.get(Prefix[i]);
            maxLen = Math.max (maxLen, len);
        }
        else {
            hm.put (Prefix [i], i);
        }
    }
    return maxLen;
}
}
```

(5) Subarray Sum equals K

Given an array of positive integers A and integer B, find and return first continuous subarray which adds to B. else -1;

```
public int [] solve (int []A, int B) {
    int n = A.length;
    long [] P = new long [n+1];
    for (int i=0; i<n; i++) {
        P[i+1] = P[i] + A[i];
    }
    HashMap <Long, Integer> map = new HashMap <>();
    for (int i=0; i<=n; i++){
        long target = P[i] - B;
        if (map.ContainsKey (target)) {
            int start = map.get(target);
            return Arrays.copyOfRange(A, start, i);
```

① Hashing - I Assignment

Q. Given 2 arrays of integers A & B, Sort A in such a way that the relative order among the elements will be the same as those are in B. For the elements not present in B, append them at last in sorted order. Return the array A after sorting from the above method.

NOTE: Elements of B are unique

↳ Solve using Comparators:

```
Public ArrayList<Integer> Solve (ArrayList<Integer> A, ArrayList<Integer> B)
    HashMap<Integer, Integer> mp = new HashMap<>();
    for (int i=0; i<B.size(); i++) mp.put(B.get(i), i)

  Collections.sort(A, (a,b) -> {
        if (mp.containsKey(a) && mp.containsKey(b))
            return mp.get(a) - mp.get(b);

        else if (mp.containsKey(a))        // if only one
            return -1;
        else if (mp.containsKey(b))
            return 1;
        else return a-b;
    });
    return A;
}
```

② # Longest Substring without Repeating Characters

- Sliding Window Technique: We use 2 pointers, L (left) and R (right)
- move r to include chars. in a HashSet.
- If you encounter a char. already in the set, move L forward and remove characters from the set until the duplicate is gone
- Complexity: O(N).

Code →

```
Public static int LOL (String s){
    HashSet <Character> hs = new HashSet <>();
    int l=0, r=0, ans = 0;
    int n = s.length();

    While (r<n){
        char ch = s.charAt(r);
        while(hs.contains(ch)){
            hs.remove(s.charAt(l));
            l++;
        }
        hs.add(ch);
        ans = Math.max(ans, r-l+1);
        r++;
    }
    return ans;
}
}
```

|| If char. exists, shrink window from the left

|| Update max length

③ # Longest Consecutive Sequence

You are given an A[](unsorted) of integers find the length of the longest set of consecutive integers. Consecutive means: $x, x+1, x+2, x+3, \ldots$ (order in array does not matter)

ex: A = [100, 4, 200, 1, 3, 2] → consecutive set: [1, 2, 3, 4] → len: 4

Sol.

```
public static in longestConsecutive(int[] nums){
    HashSet <Integer> hs = new HashSet <>();
    for (int num: nums) hs.add(num);

    int ans =0;
    for (int x : nums){
        if (!hs.contains(x-1)){
            int currNum = x;
            int len = 1;
            while(hs.contains(currNum+1)){
                currNum++;
                len++;
            ans = Math.max(ans, len);
```